02-750 Week 10

Automation of Scientific Research

Aidan Jan

March 26, 2026

What is a Hyperparameter?

Definition: A *hyperparameter* is an external configuration setting defined by the user **before** training a machine learning model, which govern the learning process itself

- Unlike parameters (e.g. weights), they are **not learned from data**, but directly affect performance, including speed of convergence and model accuracy

Some examples of hyperparameters:

Model	Hyperparameters
Linear regression	Ridge/Lasso penalties
Logistic regression	L1, L2 regularization penalties
Decision Tree	Splitting criterion, pruning cost, max depth
k-Nearest Neighbors	k, weights, metric, abstention threshold
Support Vector Machines	C, kernel, gamma
Neural Networks	Activation function, learning schedule, layer size

Hyperparameter tuning is a special case of algorithm selection in Machine Learning

Hyperparameter Tuning and Transfer

Hyperparameter tuning is important across ML

- Data prep + hyperparameter tuning take up most of the applied ML researcher hours
- Takes up to 90% of compute
- Critical in high-stakes and large-scale applications

Hyperparameter transfer is crucial today!

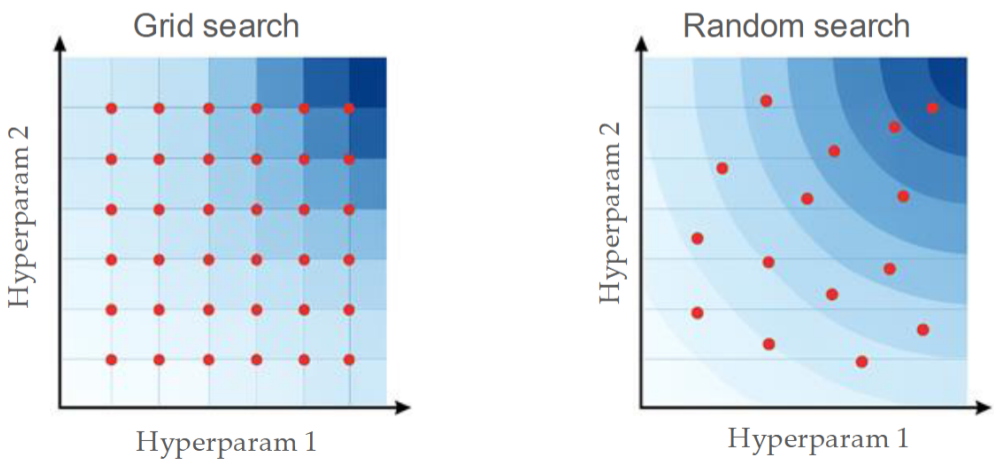
- Unavoidable in LLMs where each of the above is magnified multifold!

Algorithm Design for Machine Learning

- **Hyperparameter tuning** is poorly understood and yet of critical importance
  - why? ML works on data
  - Data → Machine Learning → Predictions
  - There is NO single best algorithm + hyperparameter!
  - Must tune / configure for the best performance on **domain-specific data**
- Current practices require incredible amounts of **compute** and **engineering** efforts, and yet with no guarantees!
- Understanding how the performance actually varies with the hyperparameter is crucial for **principled tuning**

Hyperparameter Tuning Set-Up

- Tune hyperparameters such as learning rate, batch size, weight decay
- $f(\alpha) = \text{val_loss}$
- Baselines: grid search, random search
- Blackbox optimization (zeroth-order optimization)
  - No gradient info; treat function as a "black box"



Hyperparameter Tuning as Bayesian Optimization

- Gaussian Process:
  - A collection of (infinitely many) random variables that are jointly Gaussian
  - A distribution over functions - models noisy evaluation of some $f(\alpha)$
  - Given by a mean function $m(\alpha)$ and covariance $k(\alpha, \alpha')$

$$E[f(\alpha)] = m(\alpha)$$
$$E[(f(\alpha) - m(\alpha))(f(\alpha') - m(\alpha'))] = k(\alpha, \alpha')$$

- Since all finite collections of function values are assumed jointly Gaussian, the conditional distribution of any new point given the observed points is also Gaussian, i.e., posterior predictive mean and variance at α^*, given observed points A is:

$$\mu(\alpha^*) = K(\alpha^*, A)K(A, A)^{-1}f(A)$$
$$\sigma^2(\alpha^*) = K(\alpha^*, \alpha^*) - K(\alpha^*, A)K(A, A)^{-1}K(A, \alpha^*)$$

Acquisition function

- Trade off exploration vs. exploitation

$$EI(\alpha) = E[\max(0, f_{best} - f(\alpha))]$$

Bayesian optimization:

```
for i in {0, ..., n}:
    // use GP to compute EI
    select a* = max_a EI(a)
    compute val_loss of a*
```

Example: Neural Architecture Search

Define the search space as a DAG with architecture components (e.g., conv_3x3, conv_5x5, pool, fc)

- The search space is a critical decision

Biochemical Motivation

Example 1

Optimizing High Performance Liquid Chromatography (HPLC) parameters

- Chromatography:a family of techniques used in analytical chemistry for separating, identifying, and quantifying the components in a mixture
- HPLC is faster, more sensitive, and higher resolution than traditional column chromatography, but also more expensive
- The HPLC experiment has a number of **adjustable parameters** which can affect the **resolution** of the resulting chromatogram
  - Parameters include: size and contents of column, type and concentration of the solvents, type and concentration of buffers, flow rate, temperature, mass and volume of the sample
- Let x be a vector encoding a set of HPLC parameters: $f : \mathcal{X} \to \mathbb{R}^d$ be an **unknown function** that maps x's to chromatograms, and $g : \mathbb{R}^d \to \mathbb{R}$ be a function that computes the resolution of a chromatogram

In this case, we can write the objective function as

$$x^* = \arg\max_{x \in \mathcal{X}} g\left(f(x)\right)$$

Note that $g\left(f(x)\right)$ is a black-box function, because f is a black-box function

Example 2

Designing Peptides

- Suppose that we want to design a **neoantigen vaccine** to direct a patient's immune system to attack tumor cells (**immunotherapy**)
- Let $x \in \{Ala, \cdots, Tyr\}^k$ be a peptide sequence of length k; $h : \mathcal{X} rightarrow \mathbb{R}$ be **a unknown function** that maps sequences to **therapeutic activities**
- Objective: $x^* = \arg\max_{x \in \{Ala, \cdots, Tyr\}^k} h(x)$

Note: function h is a black-box function that represents a multitude of factors, ranging from the physics of the interaction between the peptide(s) and key molecules (e.g., MHCs) to processes that occur in dendritic and T cells, which are not completely understood

Optimizing Black Box Functions

- Clearly, HPLC parameter adjustment and peptide design are *very* different tasks, but both involve optimizing unknown (**black box**) functions
  - HPLC optimization:
    $$x^* = \arg\max_{x \in \mathcal{X}} g\left(f(x)\right)$$
  - Peptide design:
    $$x^* = \arg\max_{x \in \{Ala, \cdots, Tyr\}^k} h(x)$$

- Moreover, evaluating the functions is expensive, since doing so involves running experiments, which take time and money
  - Assume that we can't afford to simply perform a brute-force search over designs
  - Instead, we will discuss strategies for optimizing black box functions that select designs to test experiments using a **surrogate model** (a.k.a., surrogate function)
    - For example, learn a regression model of the form $g' : \mathcal{X} \to \mathbb{R}$ and use it to find $\hat{x}^*$. We will **assume** that learning and evaluating g' is much faster than evaluating $g(f(x))$

Active Learning vs. Model-based Optimization

Approach	Goal	The Role of the Current Model	Query Selector Goal
Active Learning	Find a good model	Representative(s) from the current version space	Shrink the version space, using the current model(s)
Model-based Optimization	Find a good design	Point us towards better designs	Find potentially optimal designs, using the current model(s)

High-Level Query Selection Criteria: Exploration vs. Exploitation

Exploration

- Select designs that are from unexplored regions of the design space, $\mathcal{X}$
  - **Strength:** We gain knowledge that might help us build a better model (i.e., one that is better at directing us towards optimal designs)
  - **Weakness:** We may waste effort (opportunity loss)

Exploitation

- Select designs that are predicted to be optimal under the current model, h
  - **Strength:** Leverage the information we have
  - **Weakness:** We may get stuck in a local optimum (opportunity loss)

Effective model-based optimization methods make an explicit tradeoff between exploration and exploitation

Regret

We can define the regret of a query selection strategy with respect to the difference in the expected **rewards** relative to the optimal strategy

Example Scenario: Neoantigen vaccine design

- Let μ^* be the expected number of tumor cells killed per day using optimal design, d^*
- Let μ_d be the expected number of tumor cells killed per day using arbitrary design, d
- The single-day regret is $\mathbb{E}_{d \in D}[\mu^* - \mu_d]$
- The total regret (opportunity loss) after T days is $R = \Sigma_t^T \mathbb{E}_{d \in D}[\mu^* - \mu_d]$

It can be shown that:

- If we use **exploration-only**, then the regret is linear in T
- If we use **exploitation-only**, then the regret is linear in T
- *Some* strategies that balance exploration and exploitation are no-regret (i.e., sublinear in T)

Sequential Model-based Optimization

The need to optimize expensive black box functions arises in many contexts

- For example, **Hyper-parameter Optimization** in Machine Learning
  - Hyper-parameters refer to any parameter that must be set **before** learning
  - Typical examples:
    * The number of trees to use when learning a Random Forests
    * Regularization parameters (e.g., Lasso or Ridge regression)
    * The topology and size of neural networks (including deep models)
    * Learning rates and dropout rates for neural networks

Any method for optimizing expensive black box functions can also be used to solve design problems in biology and chemistry. For example, HPLC experiment configuration, or the residues in a peptide's primary sequence can be treated as hyper-parameters.

Algorithm

Generic algorithm for sequential model-based optimization (SMBO) $SMBO(f, \hat{f}_0, T, S)$

1. $D = \{\}$
2. For $t = 1$ to T:
3. $\quad x^* = optimize_x S(x, \hat{f}_{t-1})$
4. $\quad$ evaluate $f(x^*)$
5. $\quad D = D \cup \{x^*, f(x^*)\}$
6. $\quad \hat{f}_t = learn(D)$
7. Return best design in D

Here,

- f is the true model (i.e., the oracle)
- $\hat{f}_t$ is the **surrogate model**
- T is the experiment budget
- S is the **selection function** (aka., acquisition function), that defines a tradeoff between **exploring** the domain, and **exploiting** the knowledge we already have

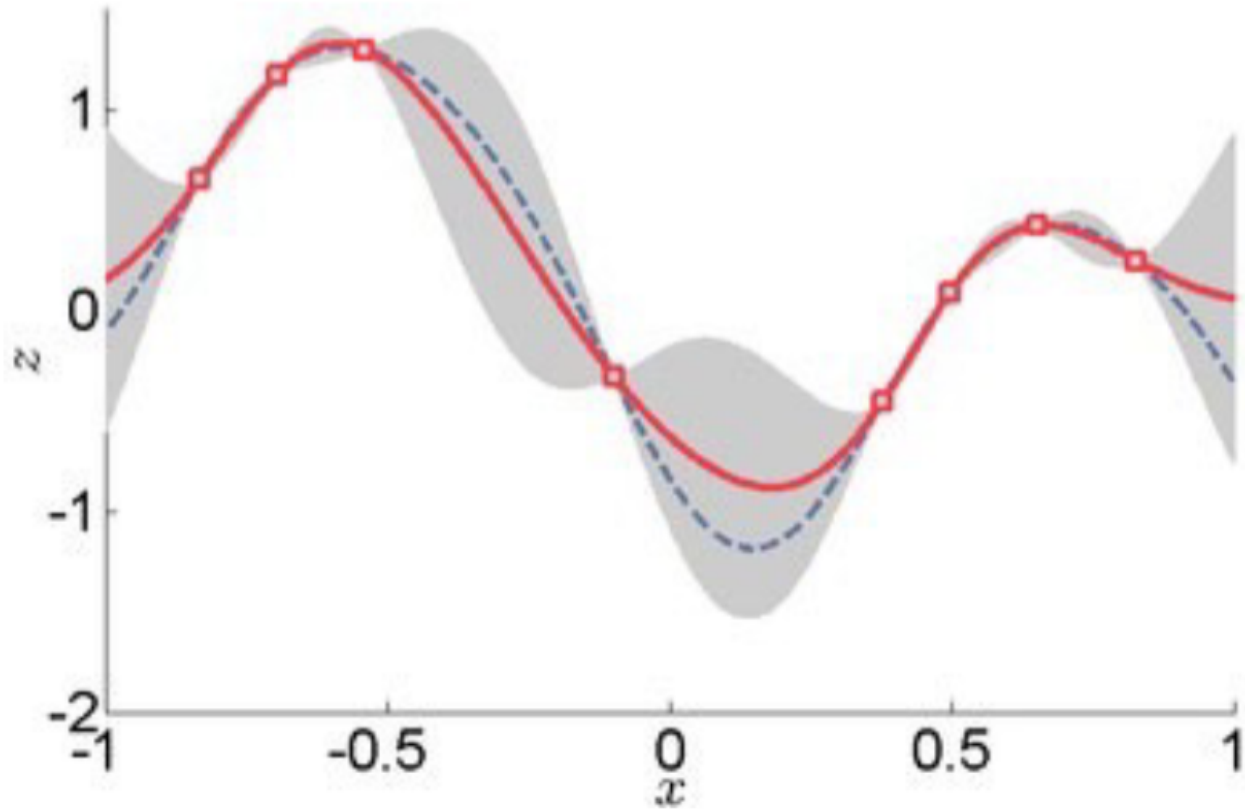
Surrogate Models

- Since we are usually trying to optimize a numeric objective (for example, chromatogram resolution or therapeutic activity), it makes sense to use a regression model, $\hat{f} : \mathcal{X} \rightarrow \mathbb{R}$, to map the adjustable (hyper) parameters, $\mathcal{X}$ to the objective
  - Naturally, $\hat{f}$ is just an approximation to the true, unknown function, f
  - Again, assume that learning and finding the (approximate) maximum of $\hat{f}$ can be performed much faster than the time needed to evaluate f
- The choice of surrogate model is partially determined by the nature of the selection function S
- If the selection function simply finds the x maximizes $\hat{f}(x)$ (that is, 100% exploitation), then we are free to use *any* regression model

- More sophisticated selection functions make a trade-off between **exploiting** the current model, and **exploring** the design space (i.e., selecting points in regions where the value $\hat{f}(x)$ is uncertain)
- Such selection functions require models that not only make predictions, but also quantify the confidence of those predictions

Optimization	**Active Learning**
Exploitation = Testing/Validation	Exploitation = Search Boundaries
Exploration = Improving model	Exploration = Search new areas

Surrogate Models: Quantifying Uncertainty



Here, the grey regions illustrate the uncertainty of the surrogate model, $\hat{f}(x)$.

- Sophisticated selection functions take both the current model (the red line) and the uncertainty into account when selecting the next design to text

Option 1: Random Forest Regression

- Random Forests (and other ensemble methods) can be used as a fast (although perhaps crude) means to quantify uncertainty using the variance of the predictions made by the trees in the forest
- Random Forests are very versatile, because they can handle mixed feature types (numeric, categorical, etc.)

Option 2: Gaussian Process Regression (aka. Kriging) Gaussian process regression (GPR) is a non-parametric, Bayesian approach to regression

- Informally, a Gaussian Process (GP) extends the concept of a **multivariate Gaussian probability distribution** over d-dimensional vectors, to a probability distribution over continuous *functions*

- Key idea: Any d-dimensional continuous function, $f : \mathbb{R}^d \to \mathbb{R}$, can be interpreted as the realization of an **infinite** number of random variables, one for each point in $\mathbb{R}^d$
- A GP is completely specified by two parameters, m and K

$$f \sim \mathcal{GP}(m, K)$$

  - $m(x) = \mathbb{E}[f(x)]$ is the **mean fucntion**
  - $K(x, x')$ is the **covariance function**, which can be used to compute the uncertainty (the grey area)
  - K is a **kernel**, that is, it is a measure of how similar points x and x' are.
- Given training data (i.e., measurements of the function at various points in the domain) and a prior, $f \sim \mathcal{GP}(m, K)$, it is possible to compute
  - The posterior distribution
  - The expected function
  - Uncertainties

Option 3: Tree-Structured Parzen Estimator (TPE)

- The Random Forest and Gaussian Process models encode $P(y|x)$, where y corresponds to the output of $f(x)$
- In contrast, the TPE method encodes the distributions $P(x|y)$ and $P(y)$
  - Recall that $P(y|x) \propto P(x|y)P(y)$ via Bayes' Theorem
- This strategy is used because it facilitates making the trade-off between exploration and exploitation and finding the best value of x

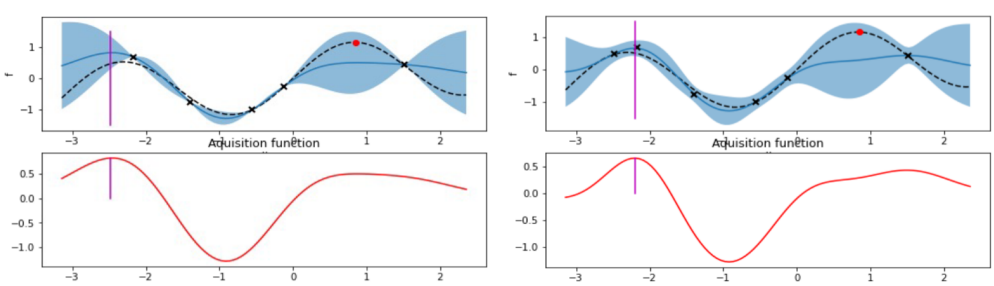
Selection (Acquisition) Functions (S)

Assuming that we have chosen a surrogate model capable of quantifying its confidence in its own predictions (e.g., Random Forests, Gaussian Processes, TPEs), there are various strategies we can use to select the next design to test

Approach 1

Select the point with the maximum of the GP mean

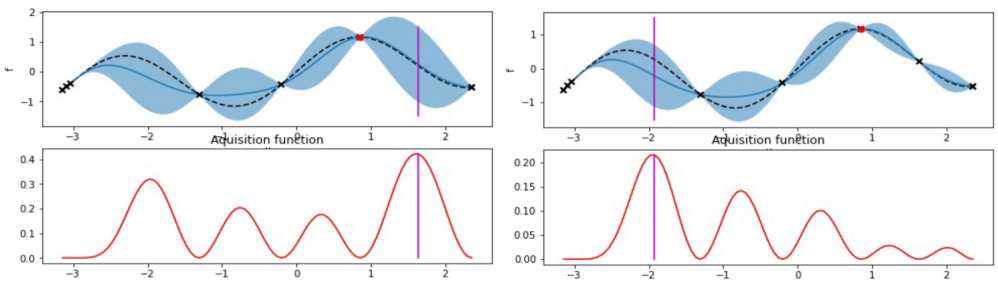
- Magenta line highlights where we want to sample next based (i.e., maximum of GP mean)
- Red dot shows the maximum of the true (unknown) function



Approach 2

Select the point with maximum of the GP variance

- Magenta line highlights where we want to sample next based (i.e., maximum of GP mean)
- Red dot shows the maximum of the true (unknown) function

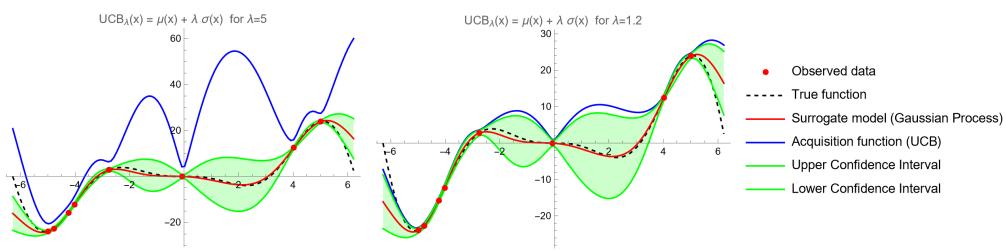


Approach 3

Upper Confidence Boudn (UCB) defined as

$$a(x; \lambda) = \mu(x) + \lambda\sigma(x)$$

- With UCB, the **exploitation vs exploration** tradeoff is straightforward and easy to tune via the parameter λ
- Concretely, UCB is a weighted sum of the expected performance captured by $\mu(x)$ of the Gaussian Process, and of the uncertainty $\sigma(x)$, captured by the standard deviation of the GP



Approach 4

Probability of Improvement (PI)

- Suppose we want to maximize $f(x)$, and the best solution we have so far is x^*. Then, we can define *improvement*, $I(x)$, as:

$$I(x) = \max(f(x) - f(x^*), 0) = \max(\mu(x) + \sigma(x)z - f(x^*), 0)z \sim \mathcal{N}(0, 1)$$

- For some point x, we assign a value $I(x)$

$$\text{PI}(x) = \Pr(I(x) > 0) \Leftrightarrow \Pr(f(x) > f(x^*))$$